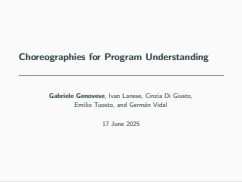
# Choreographies for Program Understanding

**Gabriele Genovese**, Ivan Lanese, Cinzia Di Giusto,
Emilio Tuosto, and Germán Vidal

17 June 2025

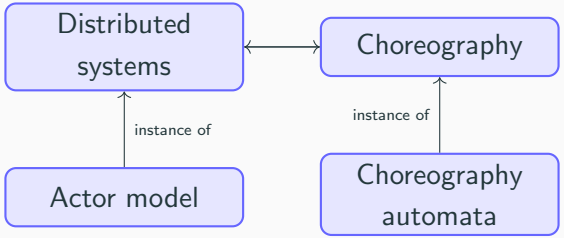





Good afternoon, everyone. I'm Gabriele Genovese, and I'm a master student of the University of Bologna. Today I'm presenting this work titled "Choreographies for Program Understanding," developed in collaboration with Professors Ivan Lanese, Cinzia Di Giusto, Emilio Tuosto, and Germán Vidal. In this work we present how we can extract global communication views from local code. This helps with understanding and verifying distributed systems, even when they weren't designed with a global model in mind.

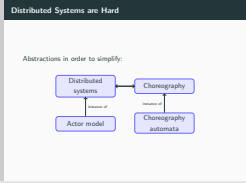
# Distributed Systems are Hard

Abstractions in order to simplify:







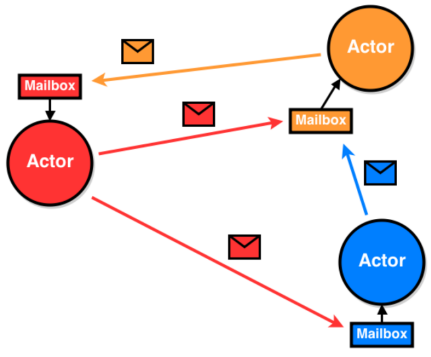




So we are in the context of distributed system, and you're probably familiar with how hard it's to create new application and protocols. In order to simplify our lives, we create some abstractions. The actor model is a very popular programming paradigm and choreographies provide a higher-level view of interactions.

# The Actor Model

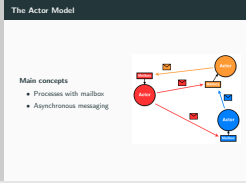
**Main concepts**

- Processes with mailbox
- Asynchronous messaging







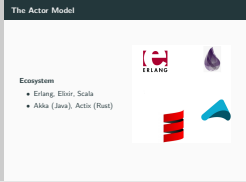




Just to be on the same I'll briefly introduce these concepts. In the actor model, we have processes, called actors, than that interacts with each other sending and receiving messages in an asynchronous network.

# The Actor Model

**Ecosystem**

- Erlang, Elixir, Scala
- Akka (Java), Actix (Rust)




The actor model is widely adopted: Erlang, Elixir, Scala are languages that implement the actor model vary well. There are also many libraries and frameworks that implement this model like Akka in Java, Actix in Rust.

# Choreography Automata [1]

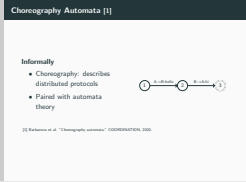
**Informally**

- Choreography: describes distributed protocols
- Paired with automata theory

[1] Barbanera et al. "Choreography automata." COORDINATION, 2020.







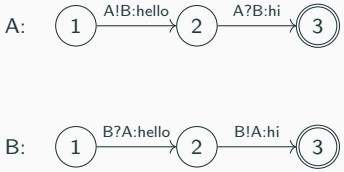

In research, it is often taken a more global view of communication. A choreography describes all the participants and their interactions from a central perspective. By pairing this with automata theory, we can rigorously define and analyze these protocols. For example, here you can see a global view...

# Global vs Local View

## Global

The communication system is seen as a whole.

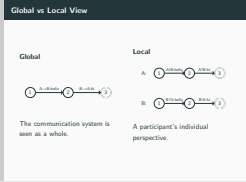
## Local




A participant's individual perspective.

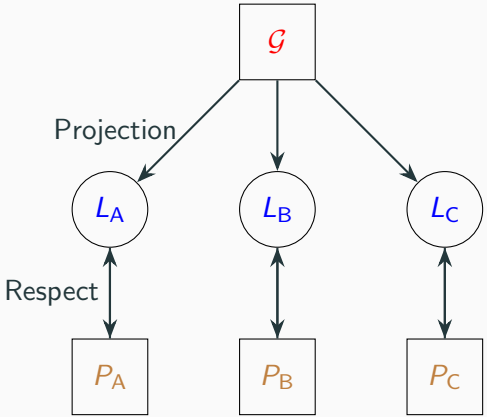







We distinguish between global and local views. The global view describes the full protocol seen from above. The local view is the individual perspective of a participant. For example, here...

# Usual practice: Top-Down Approach



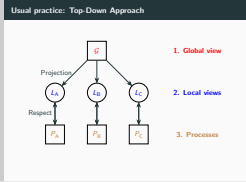

**1. Global view**

**2. Local views**

**3. Processes**








The usual practice used with this kid of frameworks is to first write the global view of the protocol, then project it into local views and implement into processes. This approach is what we call a top-down approach, because you first start by imagine the protocol, and later you implement the details

## Usual practice: Top-Down Approach

**Steps:**

1. Write the global specification
2. Project to obtain the local specifications
3. Write local programs that respect the specifications

**Problem:** difficult to integrate existing code and architecture.

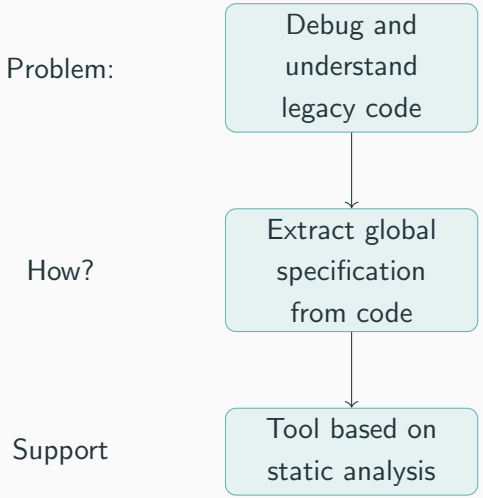




Usual practice: Top-Down Approach
Steps:
1. Write the global specification
2. Project to obtain the local specifications
3. Write local programs that respect the specifications
Problem: difficult to integrate existing code and architecture.

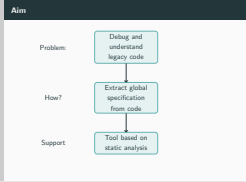
This approach is very useful because it can assure some properties over the communication. However, with this approach it's difficult to integrate existing or legacy code.

# Aim



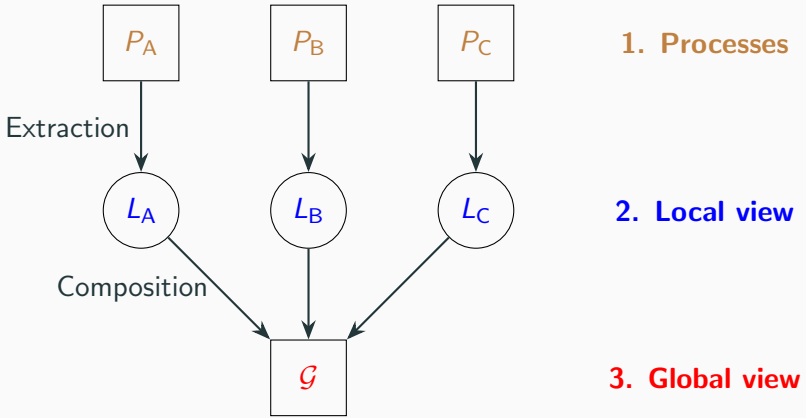




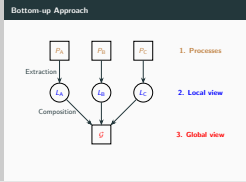
In fact, our goal is to understand and debug legacy systems. How? We think that a good way to do it is to extract a global view directly from code using a tool. This tool should be based on static analysis. This extraction algorithm is the inverse of the top-down approach explained before, this leads to a bottom-up approach.

# Bottom-up Approach








First, we analyze the code to extract local views that represents the processes. Then, we combine them to build an approximate global view of the systems.

# Bottom-up approach

**Extraction steps**

1. Analyze an input source code
2. Extract the local views
3. Compose local views to create an approximated global view

**Output:** an abstraction that captures all the possible behaviors.





Bottom-up approach
Extraction steps
1. Analyze an input source code
2. Extract the local views
3. Compose local views to create an approximated global view
Output: an abstraction that captures all the possible behaviors.

With this approach, we obtain an abstraction that should capture all the possible behavior of the system, just by analyzing the code. And with this abstraction, we can perform some analysis over it to discover some properties.

# Why

**Possible benefits**

- Improves understanding of the code
  - Give all good behavior
- Highlight bugs
  - Like deadlock, race condition, etc...





Why
Possible benefits
- Improves understanding of the code
  - Give all good behavior
- Highlight bugs
  - Like deadlock, race condition, etc...

Why, in out opinion, is this useful? First, we get a better understanding of the code, thanks of the rap presentation of all the good behaviors. We can also highlight bugs like deadlocks or race conditions.

# Requirements

- Automatic extraction
- Target mainstream languages
- Support creation and removal of participants
- Capture good behaviors and highlight misbehaviors
- Use a simple notation as output





Requirements
- Automatic extraction
- Target mainstream languages
- Support creation and removal of participants
- Capture good behaviors and highlight misbehaviors
- Use a simple notation as output

To make this useful in practice, we need several things: First, the process should be automatic, using static analysis. It should highlight both good and bad behaviors, support multiple actors, and be applicable to mainstream languages. Finally, the output should be easy to read, something people can reason with.

# Extraction steps

1. Take the file input and perform simple analysis
2. Extract a local-view for each actor found
3. Compose a global-view using the local-views
4. Minimize and analyse the output graph





Extraction steps
1 Take the file input and perform simple analysis
2 Extract a local-view for each actor found
3 Compose a global-view using the local-views
4 Minimize and analyse the output graph

To sum it up, here's an overview of our analysis pipeline: start from a file, extract the actors' local views, reconstruct a global view, and finally minimize and analyze the output.

## Problems - Undecidability

It would imply deciding termination. Choice of:

- Under-approximation: we can leave out some useful behavior
- **Over-approximation**: we include many behaviors (good or bad, present or not)



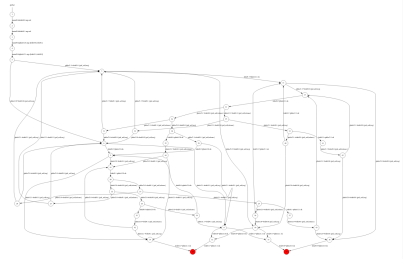


Problems - Undecidability
It would imply deciding termination. Choice of:
- Under-approximation: we can leave out some useful behavior
- Over-approximation: we include many behaviors (good or bad, present or not)

This comes obviously with some challenges. One is undecidability. We should be able to decide termination, but this is a known undecidable problem. Therefore, we must use some approximation techniques. We choose over-approximation over under-approximation because it's coherent with the requirements that we established.

# Problems - Huge descriptions

- Distributed system are naturally huge
- Needs of abstraction mechanisms: remove redundant information







Problems - Huge descriptions
- Distributed system are naturally huge
- Needs of abstraction mechanisms: remove redundant information

Another challenge is the scale of the output, that can be very huge with little local views. Distributed systems are by nature really big. So, we need abstraction and simplification techniques, like the removal or redundant information or minimization to make the output manageable and understandable.

# Motivating Examples

**Two examples:**

Dining Philosophers
(deadlock)

Bank Account
(race condition)





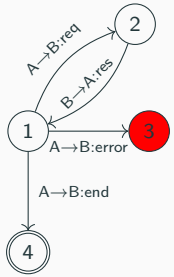
Motivating Examples
Two examples:
Dining Philosophers (deadlock)
Bank Account (race condition)

We now show two examples where this analysis helps. One is the classic dining philosophers problem, where a deadlock may occur. The other is a bank account system example that exhibits a race condition.

# Deadlock

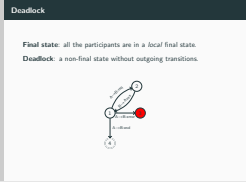
**Final state**: all the participants are in a *local* final state.

**Deadlock**: a non-final state without outgoing transitions.








A deadlock occurs when actors reach a state from which no progress is possible. In this example, A and B communicate in a way that may lead to such a blocked state. Our tool identifies this by analyzing the state machine.

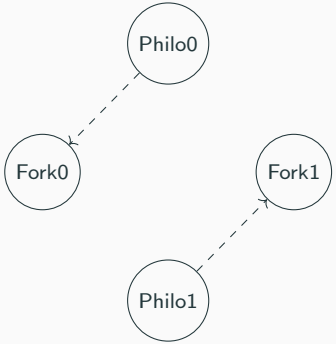
# Dining Philosophers example

```
philosopher(Fork1, Fork2) →
  send req to Fork1,
  receive ack from Fork1,
  send req to Fork2,
  receive ack from Fork2,
  eat(),
  send release to Fork1,
  send release to Fork2,
  philosopher(Fork1, Fork2).

fork() →
  receive req from Phil,
  send ack to Phil,
  receive release from Phil,
  fork().
```

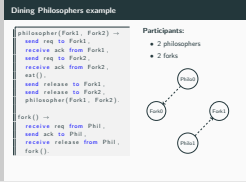
**Participants:**

- 2 philosophers
- 2 forks





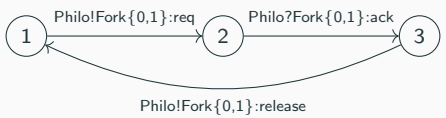


Each philosopher tries to acquire two forks before eating. If all philosophers grab one fork and wait for the other, they block. The code here expresses that logic using message-passing syntax.

# Dining Philosophers example's local views

**Note**: merge of redundant communication!

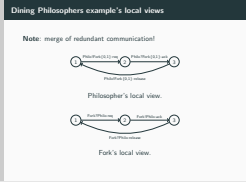



Philosopher's local view.

Fork's local view.

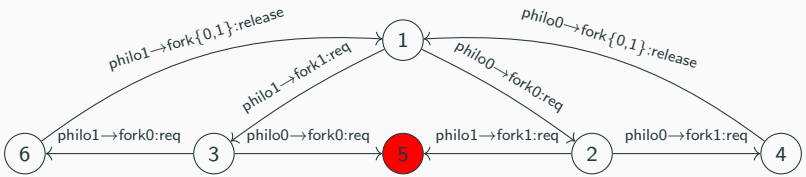







From the code, we extract the local views. The philosopher sends requests and receives acknowledgments. The fork reacts. These views are simplified to reduce redundancy and highlight structure.

# Dining Philosophers global-view




Global view of the Dining Philosophers example.

By composing the local views, we obtain this global view. We can see how deadlock arises when both philosophers wait on each other's forks. This validates the analysis and helps us fix the design.

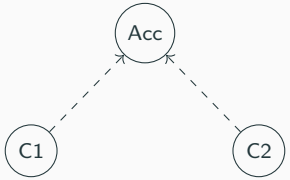
# Bank Account example

```
account(Value) →
  receive
    read from Client →
        send Value to Client,
        account(Value);
    NewValue from Client →
      account(NewValue).

client() →
  send read to Acc,
  receive Value from Acc,
  % operations on Value
  send NewValue to Acc.
```

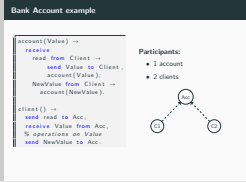
**Participants:**

- 1 account
- 2 clients



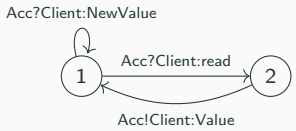




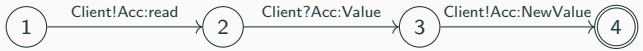




This second example models an account accessed by multiple clients. Each client reads the balance, performs operations, and sends an updated value. This leads to race conditions if not carefully synchronized.

# Bank Account example's local views



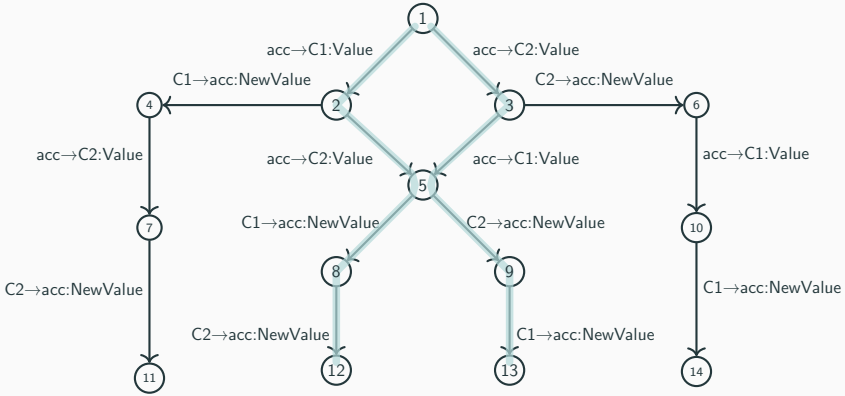

Account's local view.




Clients' local view.

Here are the local views. The account handles read and update messages. The clients interact by sending and receiving these messages. We abstract the operations and focus on communication.

# Bank Account global-view




Global view of the Bank Account example.

This global view shows interleavings of client operations. Some paths result in lost updates due to races. Others behave correctly. With this analysis, we can identify and prevent unsafe interleavings.

# Conclusion

**Summary**

- Problem: difficult to express legacy code
- Bridge the gap between programming and theoretical frameworks
- Use of bottom-up and over-approx techniques

**Future work:** create the tool as described (WIP on `github.com/gabrielegenovese/chorer`) and evaluate it on real-world examples.





Conclusion
Summary
- Problem: difficult to express legacy code
- Bridge the gap between programming and theoretical frameworks
- Use of bottom-up and over-approx techniques
Future work: create the tool as described (WIP on github.com/gabrielegenovese/chorer) and evaluate it on real-world examples.

To conclude: our work extracts global views from local code in distributed systems. We use static analysis and over-approximation to capture all behaviors. Our tool, called Chorer, is under development and will be evaluated on real systems.

# Thanks for listening!